

Decentralised Lending Service with Bitcoin Collateral

Peer to Peer Systems and Blockchains Project
Giacomo Cesaretti - 654660

1 Project structure

```
p2p-lending/
├── project2526genesis.json
├── 0xd278d247A52C550508ea2b2C9321d816238fb523psw.txt
├── chaindata_test
│   └── blk00000.dat
├── data/
│   ├── btc_balances.json
│   └── keystore/
│       └── UTC--2026-05-05T14-09-10...
├── hardhat/
│   ├── contracts/
│   │   ├── BitcoinOracle.sol
│   │   ├── LendingPool.sol
│   │   ├── Loan.sol
│   │   ├── LoanFactory.sol
│   │   ├── OwnedUpgradeProxy.sol
│   │   ├── attacks/ReentrancyAttacker.sol
│   │   └── vulnerable/VulnerableLendingPool.sol
│   ├── oracle/oracle_service.py
│   ├── scripts/
│   │   ├── initial_setup.py
│   │   ├── example_operations.py
│   │   ├── compensation_demo.py
│   │   ├── auto_approve_contributor.py
│   │   └── gasAnalysis.ts
│   ├── test/
│   │   ├── helpers/fixtures.ts
│   │   ├── BitcoinOracle.test.ts
│   │   ├── Integration.successful-flow.test.ts
│   │   ├── LendingPool.access-control.test.ts
│   │   ├── LendingPool.collateral.test.ts
│   │   ├── LendingPool.deposits.test.ts
│   │   ├── LendingPool.proposals.test.ts
│   │   ├── LendingPool.upgradability.test.ts
│   │   ├── LendingPool.withdrawals.test.ts
│   │   ├── Loan.failure-compensation.test.ts
│   │   ├── Loan.repayment.test.ts
│   │   └── ReentrancyAttack.test.ts
│   ├── deployments/
│   ├── hardhat.config.ts
│   └── package.json
```

2 Smart contract design and implementation choices

The project implements a lending protocol on a local Ethereum private chain. Contributors deposit ETH into a common funding pool, applicants submit loan proposals backed by a Bitcoin address used as liquidity proof, and contributors vote on proposals with a weight proportional to their current disposable value. Approved loans lock contributors' funds proportionally and are managed by dedicated loan contracts.

Contract architecture. The system is split into four main contracts. **BitcoinOracle** is the on-chain endpoint of the trusted oracle: it stores BTC balances in satoshi, exposes the fixed conversion rule $1 \text{ BTC} = 30 \text{ ETH}$, and accepts paid update requests. **LendingPool** contains the main service logic: deposits, withdrawals, proposal submission, voting, resolution, repayment accounting, compensation, collateral percentage updates and administrative configuration. **Loan** represents a single active loan and is deployed for each approved proposal. **LoanFactory** creates loan contracts on behalf of the pool.

Why a LoanFactory. A dedicated loan contract is required for every approved loan. Instead of instantiating each

Loan directly inside **LendingPool**, loan deployment is delegated to **LoanFactory**. This keeps the main pool focused on accounting and voting, reduces the amount of creation-code logic inside **LendingPool**, makes the deployment step explicit, and simplifies testing access control: only the configured **LendingPool** may ask the factory to create a loan.

Upgradability and termination. The main pool is used through **OwnedUpgradeProxy**. Users and scripts interact with the stable proxy address, while calls are delegated to the current **LendingPool** implementation. This satisfies the upgradability requirement while preserving the pool state across implementation changes. The proxy admin can replace the implementation, and the tests verify that non-admin accounts cannot upgrade. Loan lifecycle termination is handled explicitly through **Loan.terminate()**: successful loans can be terminated after full repayment, while failed loans can be terminated only once no locked positions remain.

Oracle fee and BTC liquidity. A BTC liquidity update request must pay at least **MIN_ORACLE_FEE**. The chosen fee is **ORACLE_UPDATE_GAS_ESTIMATE * 0.1 gwei**; the gas estimate is set to 72,270 gas, based on the measured cost of **BitcoinOracle.setBtcBalance** on the local chain. This gives a concrete implementation of the required fee rule without hard-coding an arbitrary ETH amount. The off-chain oracle can parse Bitcoin **blk*.dat** files and also supports a deterministic JSON fallback for reproducible demonstrations. The **--max-blocks 131000** option limits Bitcoin parsing to the required early mainnet range.

The two oracle modes serve different purposes. The 'blk*.dat' mode is the primary implementation: it reconstructs BTC balances from local Bitcoin block files by processing transactions and UTXOs, and it is limited to the first 131.000 mainnet blocks to keep execution and storage requirements manageable. The JSON fallback mode is not meant to replace Bitcoin parsing; it is a deterministic demo mode used when real 'blk*.dat' files are unavailable. It allows the same event-driven oracle workflow to be tested reproducibly, because requests are still emitted on-chain, observed by the off-chain service, and finalized through the trusted oracle update transaction.

Voting, locking and collateral. Proposal resolution is allowed only after **VOTING_PERIOD = 12** blocks and only by the original applicant. Votes are weighted by current disposable value; missing votes are implicit rejects and ties reject the proposal. If the proposal passes, the loaned amount is locked proportionally from contributors' disposable funds. Any precision discrepancy is deducted from the effective loaned amount, so the value sent to the applicant always matches the actual locked amount. The applicant collateral percentage starts at 50, decreases by 5 after a successful loan, and increases by 5 after a failed loan, within the interval [1, 100].

Repayment and compensation. Repayments are accepted for active loans and also for failed loans. A failed loan remains failed even if later repaid in full. Each repayment is split into principal, expected interest and overpayment. Principal progressively unlocks contributors following the specified ordering: highest initial locked amount first, with address ordering as tie-breaker. Interest is split using the collateral percentage fixed at loan creation: the collateral share is credited to the compensation pool and the remaining gain is credited as pending gains to contributors. Any overpayment beyond principal plus expected interest is credited to the compensation pool. Contributors may ask compensation multiple times for the same failed loan if the compensation pool is later refilled; any amount already compensated is forfeited from future repayments and redirected to the compensation pool.

3 Testing, gas evaluation and security discussion

The project includes a modular Hardhat test suite covering oracle permissions and fees, deposits, withdrawals, voting, collateral checks, upgradability, repayment, failed loans, repeated compensation, termination, the successful integration flow and the reentrancy demonstration. The final local run reported **40 passing tests**. A separate TypeScript script, **scripts/gasAnalysis.ts**, executes representative operations and exports **gas-analysis.csv**. It is run on the simulated Hardhat network because it uses Hardhat network helpers to mine blocks.

The final gas script records 43 transactions. It includes deployment/setup costs, the proxy upgrade path, and both cheap and expensive execution paths for proposal resolution, repayment and compensation. Table 1 reports the representative subset; the complete output is available in **gas-analysis.csv**. The most expensive deployment is the **LendingPool** implementation, because it contains the accounting, voting, repayment and compensation logic. The proxy is much cheaper than the implementation, while **upgradeTo** is cheap because it only changes the implementation pointer.

Operation / scenario	Gas used	Interpretation
Deploy <code>BitcoinOracle</code>	605,354	on-chain oracle endpoint
Deploy <code>LendingPool</code> implementation	3,379,927	main protocol logic
Deploy <code>OwnedUpgradeProxy</code>	352,025	stable address and storage
Deploy <code>LoanFactory</code>	972,307	dedicated loan deployment helper
<code>OwnedUpgradeProxy.upgradeTo</code>	30,123	implementation upgrade path
Paid BTC liquidity request	44,375	applicant pays oracle fee through pool
<code>BitcoinOracle.setBtcBalance</code>	71,680	trusted oracle balance update
Deposit, new / existing contributor	116,523 / 35,022	first registration is more expensive
Withdraw disposable value	55,432	contributor withdrawal path
Submit proposal	176,796	standard valid proposal storage
Vote approve / reject	57,842 / 57,840	vote storage cost is symmetric
Resolve rejected, low liquidity	86,869	best case: stops before loan deployment
Resolve rejected, vote majority	108,839	rejected after vote accounting
Resolve approved with <code>Loan</code> deployment	1,118,096	worst case: locking, factory call, deployment, transfer
Partial repayment	165,889	progressive principal unlocking
Final repayment after partial	274,522	closes loan as successful
Overpayment	302,960	excess credited to compensation pool
<code>Loan.markFailed</code>	115,181	marks failed and updates collateral percentage
Compensation partial / repeated / full	85,372 / 71,604 / 85,284	insufficient pool, refill claim, sufficient pool
Terminate successful / failed loan	27,337 / 58,153	failed termination requires closed positions

Table 1: Representative gas measurements from `scripts/gasAnalysis.ts`. Exact transaction fees depend on the effective gas price of the local Hardhat run; `Gas used` is the relevant comparative metric.

Reentrancy modification. The normal pool follows checks-effects-interactions and uses a reentrancy guard around ETH-moving operations. To demonstrate the vulnerability, the project includes `VulnerableLendingPool`, where the withdraw logic performs the external ETH transfer before updating the contributor state and removes the guard. `ReentrancyAttacker` deposits once, calls the vulnerable withdraw function and re-enters from its fallback/receive path. The test `ReentrancyAttack.test.ts` shows the attacker contract receiving more ETH than its own deposit and then lets the attacker owner collect the stolen funds. The secure pool avoids this by updating state before external calls and by preventing nested calls.

Possible non-reentrancy malicious strategy. A malicious contributor can try to exploit the repayment ordering rule. Since principal repayments are assigned starting from the contributors with the highest initial locked value, a contributor with large disposable funds can vote to approve a risky proposal and become the first one reimbursed by any partial repayment. If the contributor colludes with the applicant, the applicant may repay only enough principal to unlock the malicious contributor’s position, then let the loan fail. Honest contributors with smaller locked positions would remain exposed and would depend on the compensation pool. The attack is not risk-free: it requires temporary capital, sufficient voting weight, and the compensation/collateral mechanisms reduce its attractiveness. However, it can shift default losses toward honest contributors. A more robust design could distribute principal repayments pro-rata instead of by highest locked value, require stronger collateral ownership proofs, or apply risk caps per contributor/proposal.

4 Brief user manual

Install dependencies. From hardhat/ run:

```
npm install
python3 -m pip install web3 eth-account python-bitcoinlib
```

Start the private chain. From the project root:

```
geth --datadir data init project2526genesis.json
geth --datadir data --networkid 202526 --http --http.addr 127.0.0.1 \
  --http.port 8545 --http.api eth,net,web3,personal,miner,clique \
  --allow-insecure-unlock \
  --unlock 0xd278d247A52C550508ea2b2C9321d816238fb523 \
  --password 0xd278d247A52C550508ea2b2C9321d816238fb523psw.txt \
  --mine --miner.etherbase 0xd278d247A52C550508ea2b2C9321d816238fb523 \
  --nodiscover
```

Keep Geth running on `http://127.0.0.1:8545` with chain id 202526.

Compile and setup. From hardhat/:

```
npx hardhat clean
npx hardhat compile
KEYSTORE=$(ls ../data/keystore/UTC--2026-05-05T14-09-10* | head -n 1)
python3 scripts/initial_setup.py \
  --keystore "$KEYSTORE" \
  --password-file ../0xd278d247A52C550508ea2b2C9321d816238fb523psw.txt
```

This creates fresh accounts, funds them from the prefunded account, deploys the oracle, the pool implementation, the proxy and the factory, then writes `deployments/initial_setup.json`.

Run the oracle. For reproducible demos use fallback balances:

```
python3 oracle/oracle_service.py \  
  --fallback-balances ../data/btc_balances.json \  
  --from-block latest
```

If the request was emitted before starting the oracle, rerun with `--from-block 0`. For Bitcoin files, place `blk*.dat` in `../chaindata` and run:

```
python3 oracle/oracle_service.py --chaindata ../chaindata \  
  --cache ../data/btc_balances_from_blk_cache.json \  
  --bitcoin-network mainnet --max-blocks 131000 --from-block latest
```

Run demonstrations. Keep Geth and the oracle running, then use separate terminals:

```
python3 scripts/example_operations.py --btc-address btc-address-rich  
python3 scripts/compensation_demo.py  
python3 scripts/auto_approve_contributor.py --from-block latest
```

The first script demonstrates the successful loan flow. The second demonstrates failed loan marking and compensation. The third runs a contributor strategy that automatically approves new proposals.

Tests and gas analysis.

```
npx hardhat test  
npx hardhat run scripts/gasAnalysis.ts
```

The gas script produces `gas-analysis.csv`. It should be executed without `--network localgeth` because it relies on Hardhat network helpers.

More detailed commands are available in `README.md`.